

UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de Computadores 2026-2

Entrega Nº3

Del RTL al GDSII del Pochoco SoC con LibreLane y GF180MCU

Grupo 2:
Josefa Alcaíno Lira
Felipe Izcúe Ferrer
Martín Norton Haigis

Profesor de Cátedra: Jorge Gómez M.
Ayudantes:
Chen Shi Ye
Nicolás Villegas

4 de noviembre de 2026

Resumen

En este informe se documenta la implementación física del Pochoco SoC, con el procesador Espino RV32E y el juego de reflejos del Proyecto 2, desde el RTL hasta un GDSII, usando el flujo abierto LibreLane 3.0.17 y el PDK GF180MCU. Se adaptó el diseño de FPGA a ASIC mediante un módulo superior `asic_top`, se reemplazaron las construcciones propias de la FPGA (BRAM, inicialización de registros, `$readmemh`) y se comparó una ROM combinacional generada por el ensamblador frente a una memoria unificada de flip-flops. La corrida final, con $T = 100$ ns y $U = 40\,\%$, cerró con DRC, LVS y antenas limpios, y con $\mathrm{WNS}_{setup} = +37{,}22$ ns y $\mathrm{WNS}_{hold} = +0{,}17$ ns en el peor de los nueve corners, con un área de die de 0,4416 mm^2 (costo estimado US$10.156). Se barrió el período de reloj (el menor período medido que cierra en todos los corners es 70 ns, 14,3 MHz), se exploró la utilización (el área mínima que cierra entre los puntos probados es la de $U = 60\,\%$, 0,295 mm^2), se propuso un cambio al RTL que acortó la ruta crítica en 2,35 ns y se validó el netlist post-síntesis mediante simulación funcional, con trazas idénticas a las de la simulación RTL.

Índice

1. Introducción **3**

2. Entorno y reproducibilidad **4**

3. Del RTL de FPGA al RTL de ASIC **5**

4. Memoria y programa del juego **7**

5. Síntesis **9**

6. Timing **10**

7. Floorplan, red de alimentación y placement **14**

8. Signoff y layout **17**

9. Simulación funcional post-síntesis **22**

10. Conclusiones **24**

Declaración de Uso de Herramientas de Inteligencia Artificial **26**

1. Introducción

El objetivo del Proyecto 3 es llevar el Pochoco SoC [6], que integra el procesador Espino con el conjunto de instrucciones RV32E de RISC-V [5] y el juego de reflejos desarrollado en el Proyecto 2, desde RTL hasta un GDSII, empleando el flujo abierto LibreLane y el PDK GF180MCU [1, 2]. El resultado es un macro digital implementado con celdas estándar, no un chip completo: sus puertos son terminales lógicos que, en un chip real, se conectarían a pads de entrada/salida con protección y manejo de carga.

El trabajo se organiza según las actividades del enunciado [4]:

- Sección 2: entorno y reproducibilidad de la corrida final.
- Sección 3: del RTL de FPGA al RTL de ASIC (3.1).
- Sección 4: memoria y programa del juego (3.2).
- Sección 5: síntesis y trazabilidad jerárquica (3.3).
- Sección 6: timing (3.4).
- Sección 7: floorplan, alimentación y placement (3.5).
- Sección 8: signoff y layout anotado (3.6).
- Sección 9: simulación funcional post-síntesis (3.7).

Todos los grupos usan los mismos parámetros para la corrida final (Cuadro 1). Una corrida se considera cerrada cuando el flujo termina, el DRC y el LVS están limpios, no hay violaciones de antena sin justificar y se cumple WNS ≥ 0 y TNS $= 0$ tanto en setup como en hold.

Cuadro 1: Parámetros de la corrida final.

Parámetro	**Valor**
Período de reloj objetivo T (`CLOCK_PERIOD`)	100 ns (10 MHz)
Utilización de core objetivo U (`FP_CORE_UTIL`)	40 %

Fuente: Elaboración propia.

Repositorio. El código, los scripts, los resultados curados de cada corrida y el README con las instrucciones completas están en
https://github.com/F-Izcue-223/Proyecto-3-Arquitectura-de-Computadores.

2. Entorno y reproducibilidad

Las corridas se ejecutaron en un notebook con Windows, WSL2 (Ubuntu) y Docker Desktop (CPU Intel Core i5-12450H, 8 núcleos y 12 hilos; 32 GB de RAM, de los cuales WSL2 ve 15 GiB). LibreLane se usó mediante su imagen Docker oficial. El Cuadro 2 resume las versiones exactas, registradas también en `versiones_herramientas.txt` y en el README.

Cuadro 2: Versiones de herramientas, PDK y biblioteca de la corrida final.

Componente	**Versión**
LibreLane	3.0.17 (imagen `ghcr.io/librelane/librelane:3.0.17`)
PDK	`gf180mcuD`, revisión ciel `54435919abffb937387ec956209f9cf5fd2dfbee`
Biblioteca de celdas	`gf180mcu_fd_sc_mcu7t5v0`
OpenROAD [3]	build 2026-02-17, commit `dcf36133a369abc8f3c5e5738cd4d82e4903c0e0`
OpenSTA	2.7.0
Yosys	0.62 (git 7326bb7d)
Magic	8.3 rev 623
Netgen	1.5.316
Verilator	5.044 (imagen); 5.032 en el host (apt)

Fuente: Elaboración propia.

Estructura del repositorio. `rtl/` conserva sin cambios el RTL de FPGA del Proyecto 2; `rtl_asic/` contiene el RTL adaptado; `asm/` el ensamblador `pochoasm.py`; `sw/game.s` el juego; `sim/` y `sim_asic/` los testbenches; `librelane/` las configuraciones; `scripts/` los scripts de corrida; y `resultados/` las métricas y reportes de cada corrida; el GDS, el LEF y la netlist se conservan solo para la corrida final (`final_opt`).

Corrida final. La corrida entregada (`final_opt`) se ejecuta con `RUN_TAG=final_opt scripts/run_final.sh`; sin `RUN_TAG` el script crea la corrida `final`, que es la base con el RTL anterior. El script invoca LibreLane con la configuración de `librelane/`. El programa se regenera desde `game.s` con el ensamblador (Sección 4). Cada corrida completa del flujo con la estrategia 1 tomó entre 5,3 y 7,0 minutos; el ruteo detallado tomó entre 1,3 y 1,8 minutos y fue la etapa con mayor uso de CPU (pico de 945 MiB de RAM y 24 hilos).

Versión del RTL de cada corrida. Las corridas `final`, `clk_50ns` a `clk_80ns`, `util_30` a `util_60` y `final_mem_ff` se ejecutaron con el RTL **anterior** al comparador de saltos de la Sección 6, en el que la condición del salto se calculaba con la ALU. Esa versión no tiene un commit propio: el primer commit del RTL de ASIC en el repositorio (`96d69c3`, 28 de octubre de 2026) ya incluye el

comparador, y la versión anterior solo difiere de ella en el bloque de `espino_id_stage.v` citado en el Código 2. Las corridas `final_opt`, `final_slew` y `sim_corto` usan el RTL del commit `96d69c3`, que es el que está en `rtl_asic/`.

3. Del RTL de FPGA al RTL de ASIC

3.1. Módulo superior `asic_top`

Se creó `rtl_asic/asic_top.v`, que instancia `pochoco_soc` (con `DataWords = 8`) sin elementos de FPGA. Sus puertos corresponden a las funciones lógicas del juego (Cuadro 3). Incluye un sincronizador de reset de dos flip-flops, con aserción asíncrona y liberación síncrona, y sincronizadores de dos flip-flops para los cuatro botones.

Cuadro 3: Puertos de `asic_top`.

Puerto	**Dir. / ancho**	**Función**
`i_Clk`	entrada, 1	Reloj del sistema
`i_Rst_n`	entrada, 1	Reset activo en bajo (asíncrono)
`o_LED`	salida, 4	LEDs del juego
`i_Btn`	entrada, 4	Botones
`o_Seg1`, `o_Seg2`	salida, 7 c/u	Displays de 7 segmentos
`i_SPI_SCLK`, `i_SPI_MOSI`, `i_SPI_CS_n`	entrada, 1 c/u	Interfaz SPI
`o_SPI_MISO`	salida, 1	Interfaz SPI

Fuente: Elaboración propia.

```
// Asercion asincrona, liberacion sincrona
reg [1:0] rst_sync;
always @(posedge i_Clk or negedge i_Rst_n)
  if (!i_Rst_n) rst_sync <= 2'b00;
  else          rst_sync <= {rst_sync[0], 1'b1};
wire rst_n = rst_sync[1];
```

Código 1: Sincronizador de reset en `asic_top` (extracto).

3.2. Construcciones dependientes de la FPGA

Se revisó el RTL del Proyecto 2 y se ejecutaron dos corridas de diagnóstico sobre el código sin modificar: `run0` (RTL de FPGA tal cual) y `run0b` (RTL de FPGA con un modelo de BRAM para que el flujo avance). El Cuadro 4 resume cada construcción, la evidencia de LibreLane y el reemplazo.

Los logs citados están en el repositorio: `resultados/run0/logs/` (`verilator-lint.log` y `yosys-jsonheader.log`) y `resultados/run0b/logs/` (`verilator-lint.log` y `yosys-synthesis.log`); los números de línea del cuadro se refieren a esos archivos.

Cuadro 4: Construcciones dependientes de la FPGA, evidencia del flujo y reemplazo.

Construcción	**Evidencia (log:línea)**	**Reemplazo**
BRAM `SB_RAM40_4K` (`espino_register_file.v:22,33`)	`run0`: `01-verilator-lint/verilator-lint.log:1,13` (`%Error-MODMISSING`); `05-yosys-jsonheader/yosys-jsonheader.log:474`. El flujo se detiene en el paso 5.	Register file con flip-flops y lectura síncrona.
`reg [11:0] por_cnt = 12'b0` (`pochoco_soc.v:39`)	`run0b`: `verilator-lint.log:1` (`PROCASSINIT`); `yosys-synthesis.log:327`.	Reset externo con sincronizador en `asic_top`; se elimina el contador de power-on reset.
`initial` + `$readmemh` (`pochoco_ram.v:35-36`)	**Sin warning**: `yosys-synthesis.log:2311` y `:2627` no indican el problema.	ROM generada por el ensamblador; RAM de datos con flip-flops y valores de reset.
`INIT_0` de la BRAM (`espino_register_file.v:23,34`)	Solo líneas informativas.	Desaparece junto con la BRAM.
Constantes de reloj de 25 MHz en `game.s`	Sin mensaje del flujo.	Recalculadas para 10 MHz (Sección 6).

Fuente: Elaboración propia.

Construcciones sin mensaje del flujo. Dos de ellas no produjeron ningún aviso y, aun así, son un problema en silicio:

- `initial` + `$readmemh`: Yosys interpreta la inicialización como un valor de partida de la memoria, pero un ASIC no carga ningún archivo al encenderse. El programa comenzaría con contenido indeterminado. Por eso la ROM se genera como lógica combinacional a partir de `game.s`.
- Constantes de espera de 25 MHz: el flujo no conoce la semántica del programa, de modo que un reloj de 10 MHz haría que la espera de tres segundos durara siete segundos y medio.

Efecto de no corregir el RTL. Con el RTL de FPGA y un modelo de BRAM (`run0b`), la síntesis produce 68.512 celdas, 17.233 flip-flops y un área aproximada de 2,33 mm^2, es decir, más de cinco veces el área del diseño final.

4. Memoria y programa del juego

4.1. Extensión del ensamblador

Se extendió `pochoasm.py` con las opciones siguientes:

- `--rom-v`: genera la ROM combinacional en Verilog (un `case` sobre la dirección con registro de salida).
- `--init-vh`: genera la inicialización de la memoria unificada con flip-flops (estrategia 2).
- `--report`: informa las palabras de instrucciones y de datos usadas.
- `-D`: define constantes en tiempo de ensamblado, para cambiar las esperas sin editar `game.s`.

Las pruebas del ensamblador (25 en total, incluidas las de las salidas para ASIC) pasan todas.

4.2. Uso real de memoria

El número de palabras se obtuvo con `pochoasm.py --report` sobre `game.s`, cuya salida es:

```
sw/game.s -> game.hex: 114 palabras (456 de 1024 bytes)
palabras de instrucciones: 114
loads  por registro base: s0=12
stores por registro base: s0=6
```

El juego ocupa **114 palabras de instrucciones** y **0 palabras de datos**. Las 12 cargas y 6 escrituras del programa se hacen todas mediante el registro `s0`, que contiene la base de los periféricos (MMIO en `0x80000000`); ninguna accede a la RAM de datos. Ambos valores están por debajo del máximo recomendado (256 y 64 palabras). Se conservaron 8 palabras de datos para mantener una RAM funcional.

4.3. Comparación de estrategias

- **Estrategia 1:** ROM combinacional (114 palabras) generada por el ensamblador, más una RAM de datos de 8 palabras con flip-flops.
- **Estrategia 2:** memoria unificada de flip-flops de 122 palabras (114 + 8), compilada con `-DMEM_FF`.

Ambas se ejecutaron con $T = 100$ ns y $U = 40\,\%$, con el RTL base (antes del cambio de la Sección 6).

Cuadro 5: Comparación de las dos estrategias de memoria.

Métrica	**Estrategia 1**	**Estrategia 2**
Celdas tras la síntesis	5.567	21.603
Área de celdas tras la síntesis (μm^2)	162.735	654.901
Flip-flops	1.069	4.719
Área de flip-flops (μm^2)	70.251 (43,2 %)	350.780 (53,6 %)
Área de die (mm^2)	0,436	1,695
Costo estimado (US$)	10.024	38.985
Utilización tras PnR	51,4 %	54,6 %*
WNS setup, corner lento (ns)	+34,87	−740,81*
Frecuencia máxima (MHz)	15,4	1,19*
Longitud de cable	0,40 m	3,85 m*
Vías	47.733	287.608*

* Valor tomado del paso 43 (`stamidpnr`) de la estrategia 2, que **no completó** el flujo; es un valor intermedio, no un resultado de cierre.

Fuente: Elaboración propia.

Limitación de la estrategia 2. La corrida de la estrategia 2 **no se completó**: se detuvo en el paso 43 (`stamidpnr`) con una red con violación de antena, y el ruteo detallado quedó estancado en una violación de DRC entre las iteraciones 44 y 60. Los valores marcados con asterisco en el Cuadro 5 (utilización, WNS, frecuencia máxima, longitud de cable y vías) corresponden a ese paso intermedio, por lo que son indicativos y no son un cierre de timing. Esta corrida no se repitió con otra utilización, de modo que la comparación se basa en estos valores parciales y la conclusión (la ROM combinacional es la opción viable) no depende de un cierre de la estrategia 2.

Análisis. La ROM combinacional ocupa 114 palabras de lógica, que Yosys optimiza como funciones booleanas compartidas, mientras que la memoria unificada necesita 32 flip-flops por cada una de las 122 palabras más sus multiplexores de lectura y escritura. La diferencia de ∼3.650 flip-flops explica la mayor parte de la diferencia de área (3,9× en celdas, 4,0× en área de celdas y 3,9× en costo) y, a través de los cables más largos (casi diez veces la longitud de ruta, valor del paso 43), la caída de la frecuencia máxima. Por esta razón se eligió la estrategia 1 para la corrida final.

5. Síntesis

Los valores de esta sección provienen del reporte de estadísticas jerárquicas de Yosys de la corrida final (`resultados/final_opt/sintesis/stat.rpt`).

5.1. Celdas más usadas

El diseño sintetizado de la corrida final tiene 5.638 celdas con un área total de 164.956 μm^2. El Cuadro 6 lista las cinco celdas más usadas, ordenadas por cantidad como pide el enunciado, con su área y el módulo donde están.

Cuadro 6: Cinco celdas más usadas (por cantidad) tras la síntesis de la corrida final (`final_opt`).

Celda	Cant.	Área (μm^2)	% del área	Dónde están (cantidad por módulo)
dffq_1	870	55.400	33,6	Register file 544; resto del diseño 326
nand2_1	825	9.055	5,5	Resto 649; register file 88; ALU 82; contador 6
mux2_2	751	24.700	15,0	Register file 622; resto 129
oai21_1	442	7.760	4,7	Resto 359; ALU 51; register file 32
nor2_1	353	4.650	2,8	Resto 219; ALU 80; register file 46; contador 8

Fuente: Elaboración propia.

"Resto" es la lógica del módulo superior fuera de los tres bloques que Yosys mantiene separados (control, decodificación, ROM, RAM y periféricos). La lista completa de celdas está en el `stat.rpt`. Por área, la celda más grande es `dffq_1` (33,6 %) y le siguen `mux2_2` (15,0 %) y `dffrnq_1` (14.600 μm^2, 196 celdas); los flip-flops en total (1.069) ocupan 70.251 μm^2, un 42,6 % del área sintetizada.

5.2. Módulo con mayor área y contador de ciclos

Se mantuvo la jerarquía con `SYNTH_KEEP_HIERARCHY_MODULES`, de modo que el reporte de Yosys entrega el área por módulo (Cuadro 7).

Cuadro 7: Área por módulo tras la síntesis de la corrida final (`final_opt`).

Módulo	Área (μm^2)	% del total
espino_register_file	73.656	44,7
espino_alu	10.006	6,1
pochoco_cycle_counter	3.943	2,4
Resto (control, decodificación, ROM, RAM, periféricos)	77.352	46,9
Total	**164.956**	**100**

Fuente: Elaboración propia.

El módulo con mayor área es el **register file** (44,7 %), con 544 flip-flops `dffq`. El **contador de ciclos** agregado en el Proyecto 2 aporta 3.943 μm^2, que corresponde al **2,4 %** del área sintetizada y usa 32 flip-flops `dffrnq`.

Trazabilidad jerárquica. Los nombres de instancia se conservan hasta el layout, por lo que las celdas de cada bloque se identifican por prefijo: `u_soc.u_core.u_rf/` (register file), `u_soc.u_core.u_id.u_alu/` (ALU) y `u_soc.u_periph.u_cycle_counter` (contador de ciclos). Estos prefijos se usan en la Sección 8 para anotar el layout.

5.3. Comparación con lo esperado

Solo se compara el register file, porque es el único bloque para el que se puede calcular el área esperada directamente desde el RTL; no se hizo una estimación previa para la ROM ni para el total, por lo que no se comparan. En el RTL de ASIC el register file guarda 15 registros de 32 bits (`x1` a `x15`; `x0` no se almacena) y tiene lectura síncrona con dos puertos de salida registrados, por lo que se esperan $15 \cdot 32 + 2 \cdot 32 = 544$ flip-flops. Con 63,7 μm^2 por `dffq_1`, área unitaria que entrega el reporte (55.400 μm^2 entre 870 celdas), eso da $\approx$34.650 μm^2. A esto se suman los multiplexores de escritura (un `mux2` de 32,9 μm^2 por cada uno de los 480 bits de almacenamiento, $\approx$15.800 μm^2) y los de lectura (dos puertos de 32 bits, cada bit un multiplexor de 15 a 1 de $\approx$5 `mux4` de 70,2 μm^2, $\approx$22.500 μm^2). La suma, $\approx$72.950 μm^2, coincide con los 73.656 μm^2 del reporte (diferencia de 1 %), y el recuento de 544 flip-flops `dffq` es exacto.

El resultado muestra el peso de la lógica de selección: el register file ocupa más del doble de lo que ocuparían solo sus flip-flops, porque la lectura y la escritura con multiplexores agregan casi la misma área que el almacenamiento.

6. Timing

6.1. Barrido del período de reloj

Se barrió `CLOCK_PERIOD` en cinco puntos (50, 60, 70, 80 y 100 ns), todos con $U = 40\,\%$ y el RTL base. El corner típico es `nom_tt_025C_5v00` y el más lento `max_ss_125C_4v50`, de entre los nueve corners de STA del PDK.

Cuadro 8: Barrido de `CLOCK_PERIOD`: WNS y TNS de setup y hold (ns), corner típico y lento.

	Setup				**Hold**			
	WNS		TNS		WNS		TNS	
T **(ns)**	típ.	lento	típ.	lento	típ.	lento	típ.	lento
100	+65,76	+34,87	0	0	0,37	0,94	0	0
80	+45,76	+14,87	0	0	0,37	0,94	0	0
70	+35,76	+4,87	0	0	0,37	0,94	0	0
60	+25,76	−5,13	0	−444,56	0,37	0,94	0	0
50	+16,09	−14,46	0	−6.568,72	0,37	0,94	0	0

Fuente: Elaboración propia.

El TNS del corner típico es 0 en todos los puntos, y el de hold es 0 en ambos corners (valores tomados de `metrics.json` de cada corrida). El menor período medido que cierra timing en los corners exigidos es **70 ns** (14,3 MHz). Extrapolando linealmente el slack del punto de 100 ns, el período mínimo estimado es 65,1 ns (15,4 MHz); esta cifra es una estimación, porque no se corrió el flujo en ese punto.

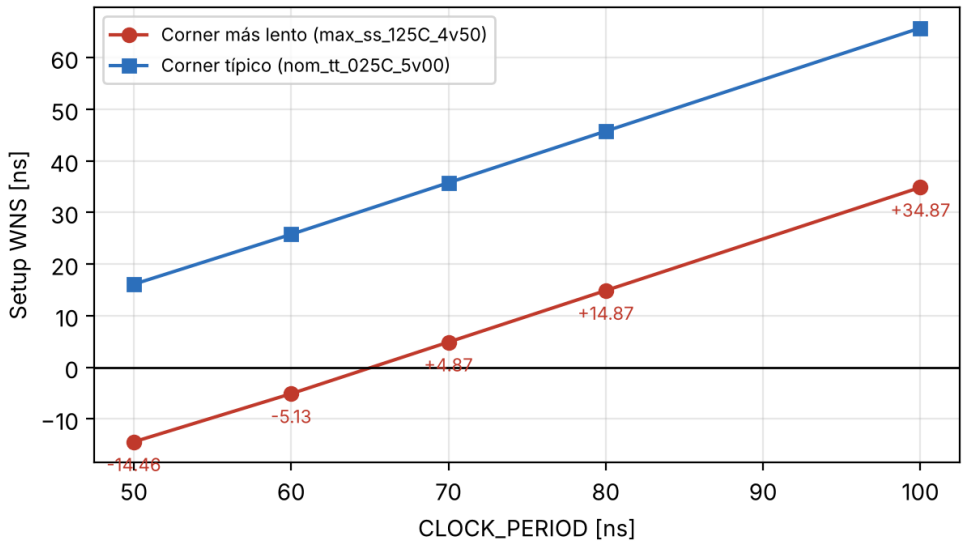


Figura 1: WNS de setup frente al período de reloj en los corners `max_ss` y `nom_tt`.

Fuente: Elaboración propia.

Advertencia sobre la optimización. Cada valor de `CLOCK_PERIOD` puede producir una nueva optimización del circuito (buffering, dimensionamiento, placement), con cambios en área y ruteo. En este barrido el número de celdas y el área fueron idénticos entre 100 y 60 ns, y solo cambiaron en 50

ns. Una explicación probable, no verificada con una corrida adicional, es que el *resizer* repara únicamente en el corner `nom_tt`, donde el slack se mantiene positivo, y por eso no reacciona al período; esto es coherente con que el corner por defecto del flujo sea `DEFAULT_CORNER = nom_tt_025C_5v00` (README); los incumplimientos en `max_ss` quedan sin reparar.

El hold no depende del período, ya que depende de los retardos entre flip-flops del mismo flanco, y por eso es constante en la tabla. El peor hold sobre los nueve corners es +0,13 ns (corrida `final`) y +0,17 ns (corrida `final_opt`).

6.2. Ruta crítica

En la corrida `final` (RTL base, $T = 100$ ns, corner `max_ss`) la ruta crítica tiene 40 celdas y un tiempo de llegada de 67,19 ns (35,26 ns en `nom_tt`):

- **Celda de inicio:** `_5373_` (`dffq_1`), que guarda `u_soc.instr_rdata[3]`. **Módulo RTL de inicio:** `pochoco_imem` (instancia `u_mem.u_imem`, en `rtl_asic/gen/game_rom.v`): es el registro de salida `rdata_o` de la ROM de instrucciones.
- **Celda de fin:** `_5228_` (`dffrnq_1`), que guarda `u_soc.instr_addr[23]` (el contador de programa). **Módulo RTL de fin:** `espino_if_stage`, registro `pc_q` (`instr_addr_o = pc_q`).
- **Función microarquitectónica:** decodificación → ejecución (comparación en la ALU) → resolución del salto → cálculo del próximo PC (*fetch*).
- **Composición:** 36,6 ns corresponden a celdas `dlyb_1` (retardo insertado como buffer) y cerca de 14 ns a la ALU.

6.3. Cambio propuesto al RTL

La ruta pasa por la ALU únicamente para evaluar la condición del salto. Se propuso calcular la condición en un comparador dedicado dentro de `espino_id_stage.v`, en paralelo con la ALU, para sacar la comparación de la ruta hacia el PC.

```
wire [2:0] br_f3  = instr_rdata_i[14:12];
wire br_eq  = (rf_rdata_a_i == rf_rdata_b_i);
wire br_lt  = ($signed(rf_rdata_a_i) < $signed(rf_rdata_b_i));
wire br_ltu = (rf_rdata_a_i < rf_rdata_b_i);
reg  br_cond;
always @* begin
  case (br_f3)
    3'b000:  br_cond =  br_eq;   // BEQ
    3'b001:  br_cond = ~br_eq;   // BNE
    3'b100:  br_cond =  br_lt;   // BLT
    3'b101:  br_cond = ~br_lt;   // BGE
```

```
    3'b110:  br_cond =  br_ltu;  // BLTU
    3'b111:  br_cond = ~br_ltu;  // BGEU
    default: br_cond =  1'b0;
  endcase
end
assign branch_taken_o = is_branch & br_cond;
```

Código 2: Comparador de saltos dedicado en `espino_id_stage.v` (extracto).

El cambio se corrió con $T = 100$ ns y $U = 40\,\%$ (corrida `final_opt`). El Cuadro 9 compara ambas corridas.

Cuadro 9: Efecto del comparador dedicado ($T = 100$ ns, $U = 40\,\%$).

Métrica	**`final` (base)**	**`final_opt`**
WNS setup, `max_ss` (ns)	+34,87	+37,22
WNS setup, `nom_tt` (ns)	+65,76	+67,04
Período mínimo estimado, `max_ss` (ns)	65,13	62,78
Frecuencia máxima estimada, `max_ss` (MHz)	15,4	15,9
Área de celdas estándar (μm^2)	207.861	209.936
Instancias de celdas estándar	9.464	9.535
Área de die (mm^2)	0,4358	0,4416
Costo estimado (US$)	10.024	10.156

Fuente: Elaboración propia.

El slack mejoró en **2,35 ns** (6,7 %) a costa de un **+1,0 %** en el área de celdas (71 celdas adicionales) y +1,3 % en el área del die. En la nueva corrida la ruta crítica pasa a ser decodificación → ALU → *write-back* (inicio en `_5516_`, `instr_rdata[4]`, el registro de salida de la ROM en `pochoco_imem`; fin en un flip-flop del módulo `espino_register_file`, `u_soc.u_core.u_rf/_2755_`, con 64,75 ns de llegada), es decir, la ruta del PC dejó de ser la peor. La mejora es modesta porque la ruta crítica sigue dominada por las celdas de retardo `dlyb_1` (622 en el netlist final), que el *resizer* usa como buffers. Solo el punto de 100 ns se ejecutó con el RTL modificado; el barrido de la tabla anterior se hizo con el RTL base.

6.4. Constantes de `game.s` para $T = 100$ ns

El juego se escribió para un reloj de 25 MHz. Con $T = 100$ ns la frecuencia es $f = 1/T = 10$ MHz, por lo que cada constante en ciclos se recalcula como $N = t/T$ para mantener el mismo tiempo en segundos:

$$N = \frac{t}{T} = t \cdot f, \qquad f = 10 \text{ MHz}. \tag{1}$$

Cuadro 10: Constantes de espera de `game.s` para 25 MHz y 10 MHz.

Constante	**Tiempo**	N **a 25 MHz**	N **a 10 MHz**
`WAIT_CYCLES` (espera de 3 s)	3 s	75.000.000	30.000.000
`TENTH_CYCLES` (una décima)	0,1 s	2.500.000	1.000.000
`DEBOUNCE_CYCLES`	20 ms	500.000	200.000
`BLINK_CYCLES`	0,25 s	6.250.000	2.500.000

Fuente: Elaboración propia.

Al pasar de 25 a 10 MHz cambian las cuatro constantes de espera del cuadro; `ROUNDS` (10 rondas) y `ERROR_CODE` (`0xEE`) no cambian, porque no dependen de la frecuencia del reloj. Por ejemplo, WAIT_CYCLES = 3 s/100 ns = $3 \cdot 10^7$ y TENTH_CYCLES = 0,1 s/100 ns = 10^6. El programa se regeneró con el ensamblador pasando estas constantes mediante `-D`. Los valores de la columna de 25 MHz son los equivalentes para el reloj de la Go Board ($N = t \cdot 25$ MHz), calculados con los mismos tiempos.

7. Floorplan, red de alimentación y placement

7.1. Exploración de la utilización

Se corrió el flujo completo con $U = 30, 40, 50$ y $60\,\%$ ($T = 100$ ns, RTL base). Todas las corridas completaron el flujo con DRC de ruteo, DRC de Magic y LVS en cero.

Cuadro 11: Barrido de `FP_CORE_UTIL`.

U **(%)**	**Die** (mm^2)	**Costo** (US$)	**WNS setup** lento (ns)	**DRC** it. 0	**Antenas**	**Util.** tras PnR
30	0,576	13.245	+41,74	0	1 red	39,3 %
40	0,436	10.024	+34,87	177	0	51,4 %
50	0,3515	8.083	+38,72	260	0	63,4 %
60	0,295	6.784	+36,82	97	0	75,5 %

Fuente: Elaboración propia.

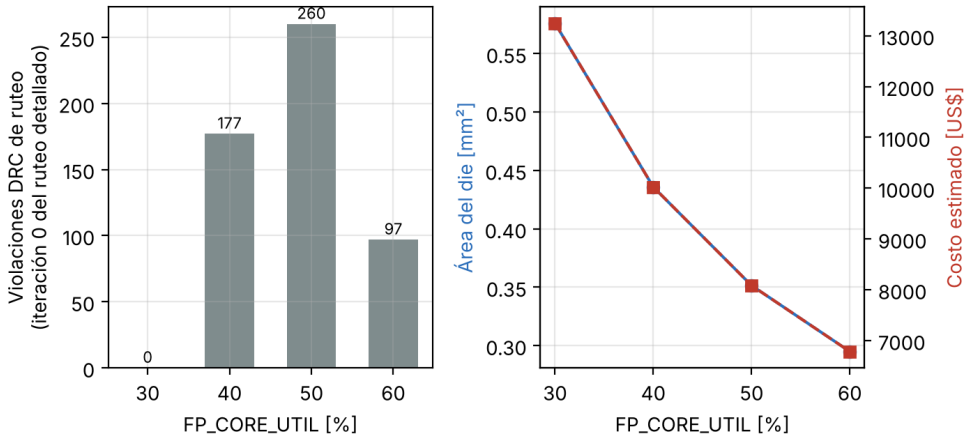


Figura 2: Utilización de core frente a violaciones de DRC en la iteración 0 del ruteo detallado y al área del die.

Fuente: Elaboración propia.

Todas las utilizaciones exploradas completaron el flujo con los criterios de cierre, por lo que el **área mínima que cierra** entre los puntos probados es la de $U = 60\,\%$ (0,295 mm^2, US$6.784). El número de violaciones de DRC al comienzo del ruteo (iteración 0) no es monótono: 0, 177, 260 y 97; el ruteador detallado las eliminó en todos los casos. La corrida final exigida por el enunciado se hizo con $U = 40\,\%$.

7.2. Ubicación de los terminales

Los terminales se agruparon por función y por borde del die (`informe/pines_justificacion.md`):

- **Oeste:** `i_Clk` e `i_Rst_n`, las señales globales de reloj y reset.
- **Sur:** botones (`i_Btn`) y LEDs (`o_LED`), la interacción directa con el jugador.
- **Norte:** `o_Seg1` y `o_Seg2`, los dos displays de 7 segmentos.
- **Este:** la interfaz SPI (esclavo SPI), alejada del reloj.

Agrupar los pines de cada función acorta los cables entre el macro y sus periféricos y reduce el cruce de rutas. Estos terminales son puntos de conexión lógicos del macro; un chip completo necesitaría además pads de I/O con protección ESD, buffers de potencia para manejar los LEDs y los displays, y niveles de voltaje adecuados.

7.3. Costo estimado

Con el dato visto en clases (US$23.000 por 1 mm^2) y suponiendo que el costo escala linealmente con el área, la corrida final (die de 0,4416 mm^2) cuesta aproximadamente

$$0{,}4416\ \text{mm}^2 \times 23.000\ \text{US\$/mm}^2 \approx \text{US\$10.156}. \tag{2}$$

Limitaciones. El costo real no escala linealmente con el área: depende del rendimiento (*yield*), que cae con el área; de los costos fijos de máscaras y de las corridas multi-proyecto (MPW), que tienen un tamaño mínimo; del encapsulado, las pruebas y los pads de I/O que aquí no se consideran; y del volumen. También usamos el área del die, no la de las celdas.

Decisión con mayor impacto. La elección de la estrategia de memoria tuvo el mayor impacto: pasar de la ROM combinacional a la memoria unificada de flip-flops multiplicó el área del die por 3,9 (de 0,436 a 1,695 mm^2) y el costo estimado de US$10.024 a US$38.985. El área y el costo de la estrategia 2 vienen de una corrida que no se cerró (se detuvo en el paso 43), por lo que son los del die fijado en el floorplan y no los de un diseño terminado. En comparación, variar la utilización entre 30 y 60 % cambió el costo en un factor de 2,0, y el cambio de RTL de la Sección 6 lo cambió en 1,3 %.

8. Signoff y layout

8.1. Resultados finales

Cuadro 12: Resultados de signoff de la corrida final (`final_opt`, $T = 100$ ns, $U = 40\,\%$).

Verificación	**Resultado**
DRC de Magic	0 violaciones
LVS (Netgen)	0 diferencias
DRC de ruteo (OpenROAD)	0 violaciones
Violaciones de antena	0
Setup, WNS `max_ss` / `nom_tt` (ns)	+37,22 / +67,04
Setup, TNS	0
Hold, WNS `nom_tt` / `max_ss` (ns)	+0,44 / +1,06
Hold, peor de los nueve corners (ns)	+0,17
Hold, TNS	0
Violaciones de *slew* máximo / capacitancia máxima	2.426 / 7
Longitud de cable (μm) / vías	397.054 / 47.238
Potencia estimada (mW)	17,8

Fuente: Elaboración propia.

DRC. El DRC de KLayout se omite en el flujo para el PDK `gf180mcuD`, que no incluye un *runset* para esta herramienta; el DRC válido es el de Magic, que resultó limpio.

Violaciones de *slew* y capacitancia. Las 2.426 violaciones de *slew* máximo aparecen en el corner `max_ss` y son avisos de la verificación de integridad de señal, no un criterio de cierre del enunciado. Se deben a que el corner lento (125 °C, 4,5 V) degrada los flancos y a que las celdas de retardo `dlyb_1`, que el *resizer* usa como buffers, tienen poca fuerza de excitación. Para comprobarlo se hizo una corrida adicional (`final_slew`, mismo RTL, $T = 100$ ns y $U = 40\,\%$) que excluye del *resizer* las celdas `dly*` y `buf_1`, y fija `PL_RESIZER_SETUP_SLACK_MARGIN = 10`. El Cuadro 13 compara ambas corridas.

Cuadro 13: Efecto de excluir las celdas `dly*` y `buf_1` del *resizer*.

Métrica	`final_opt`	`final_slew`
Violaciones de *slew* máximo	2.426	443
Violaciones de capacitancia máxima	7	20
WNS setup, `max_ss` (ns)	+37,22	+56,56
WNS setup, `nom_tt` (ns)	+67,04	+74,49
WNS hold, peor de los nueve corners (ns)	+0,17	+0,17
TNS setup / hold	0 / 0	0 / 0
Área de celdas estándar (μm^2)	209.936	197.375
Longitud de cable (μm)	397.054	385.011
Potencia estimada (mW)	17,8	17,3
DRC de Magic / LVS / antenas	0 / 0 / 0	0 / 0 / 0

Fuente: Elaboración propia.

Las violaciones de *slew* bajaron un 82 %, el slack de setup en el corner lento mejoró en 19,3 ns y el área de celdas bajó un 6 %, con el signoff igualmente limpio. A cambio, las violaciones de capacitancia máxima subieron de 7 a 20. Esta configuración no se aplicó al resto de las secciones (síntesis, barridos y layout anotado), de modo que la corrida final del informe sigue siendo `final_opt`.

8.2. Violación de antena y corto en Metal2: casos documentados

Los pasos 40, 42, 44 y 46 de esta sección son de la corrida final (`final_opt`). Tras el ruteo global, el reporte de antenas (paso 40) listó **14 redes con violación**: nueve son bits del bus de escritura del register file (`u_soc.u_core.rf_wdata[1]`, `[9]`, `[10]`, `[13]`, `[19]`, `[21]`, `[22]`, `[29]` y `[31]`) y cinco son otras redes (`net596`, `net734`, `u_soc.u_core.u_rf/_0634_`, `u_rf/net288` y `u_rf/net68`). Los pines receptores eran entradas de los buffers de *fanout* del register file; por ejemplo, `u_soc.u_core.u_rf/fanout80/I` y `fanout81/I` (red `rf_wdata[10]`, en (136,6; 482,2) y (154,6; 474,3) μm) y `fanout74/I` (red `rf_wdata[13]`, en (262,6; 478,2) μm). Una violación de antena ocurre cuando un cable largo de metal, conectado solo a la compuerta de un transistor durante la fabricación, acumula carga del plasma y puede dañar el óxido.

Los pines de las 14 redes están repartidos por todo el die (entre $x \approx 32$ y 600 μm y entre $y \approx 153$ y 627 μm, Figura 3), no concentrados en un bloque. El paso 42 (reparación de antenas) las resolvió con dos mecanismos:

- *jumpers*: cambio de capa de metal del tramo largo, de forma que la compuerta quede conectada por un tramo corto durante la fabricación de cada capa;
- diodos de antena: celdas `ANTENNA_*` conectadas a la red, que derivan la carga al sustrato.

El reporte del paso 46 quedó vacío, es decir, sin violaciones residuales. El DEF final contiene 8 instancias `ANTENNA_*` (por ejemplo, `ANTENNA_8` en (57,7; 541,0) μm). Como se corrigieron, no queda ninguna violación que requiera justificación.

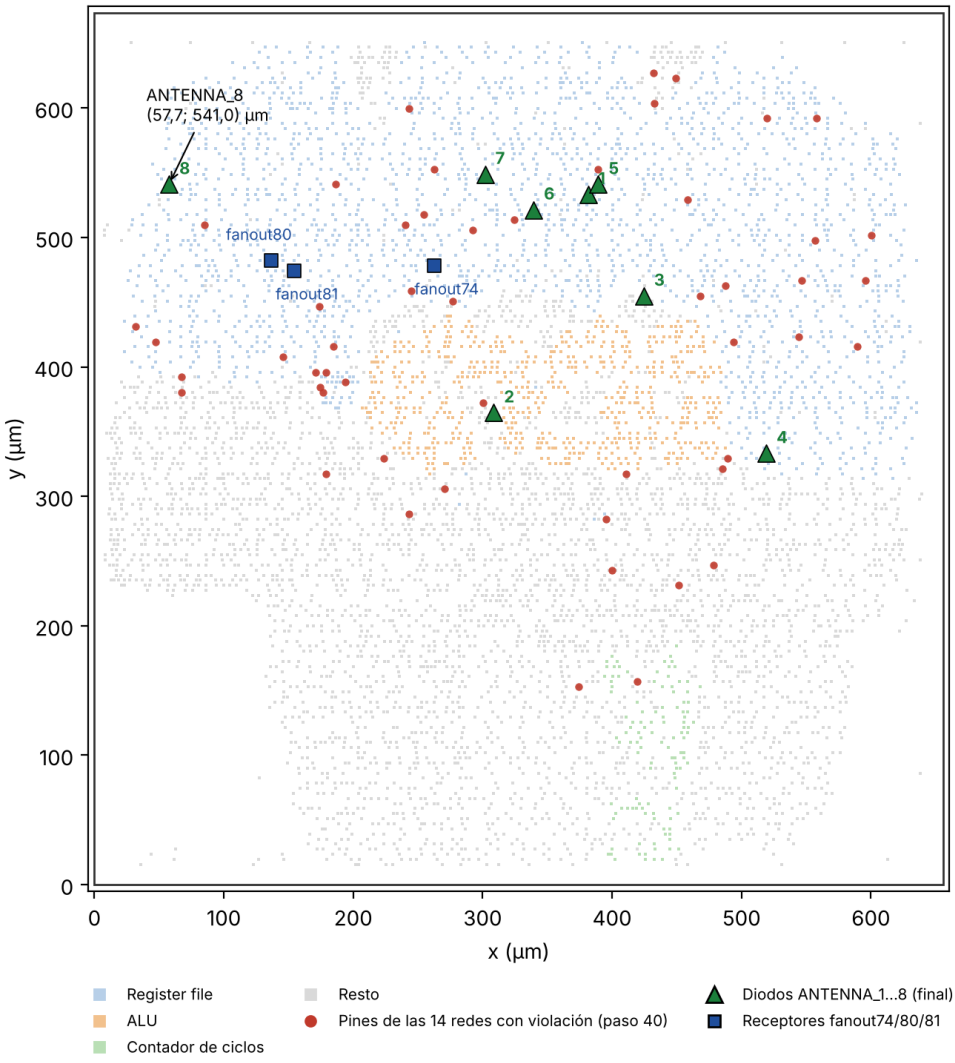


Figura 3: Ubicación de los pines de las 14 redes con violación de antena del paso 40, de los receptores `fanout74`, `fanout80` y `fanout81` y de los ocho diodos `ANTENNA_*` del DEF final (`final_opt`). Los colores de las celdas indican el módulo (register file, ALU, contador de ciclos).

Fuente: Elaboración propia.

Corto en Metal2 durante el ruteo detallado. Este caso no es de antena sino de DRC. En el paso 44 (ruteo detallado) apareció una violación de DRC: tras la iteración de optimización 3 quedó **1 violación, un corto (*short*) en Metal2**, luego de bajar desde 2.004 violaciones en la

iteración 0 (669 y 545 en las iteraciones 1 y 2). La iteración siguiente (4, de *guides tiles*) la eliminó y el ruteo terminó con 0 violaciones. Luego, el chequeo de antenas del mismo paso encontró 5 redes con violación, y se ejecutó la iteración 1 de reparación de antenas, que volvió a rutear (100, 16 y 13 violaciones de DRC, y 0 al final). La verificación de antenas posterior dio 0 redes con violación. Un corto en una capa de metal habría sido un defecto grave en el chip, pero aquí fue un estado transitorio del ruteador, corregido antes de terminar el paso, y la verificación independiente de Magic (DRC) y la comparación de Netgen (LVS) resultaron limpias (Cuadro 12).

8.3. Layout anotado

Se identificaron los tres bloques con los nombres jerárquicos conservados (Sección 5). El Cuadro 14 entrega la extensión de cada uno y la Figura 4 muestra el layout anotado en KLayout.

Cuadro 14: Extensión de los bloques en el layout (coordenadas en μm).

Bloque	**x**	**y**	**Celdas**
Register file	23–635	314–651	1.845
ALU	207–488	318–439	580
Contador de ciclos	394–463	20–184	114

Fuente: Elaboración propia.

Las cajas son envolventes (el rectángulo mínimo que contiene todas las celdas del módulo), por eso la de la ALU cae dentro de la del register file: las celdas de ambos módulos están mezcladas en esa zona.

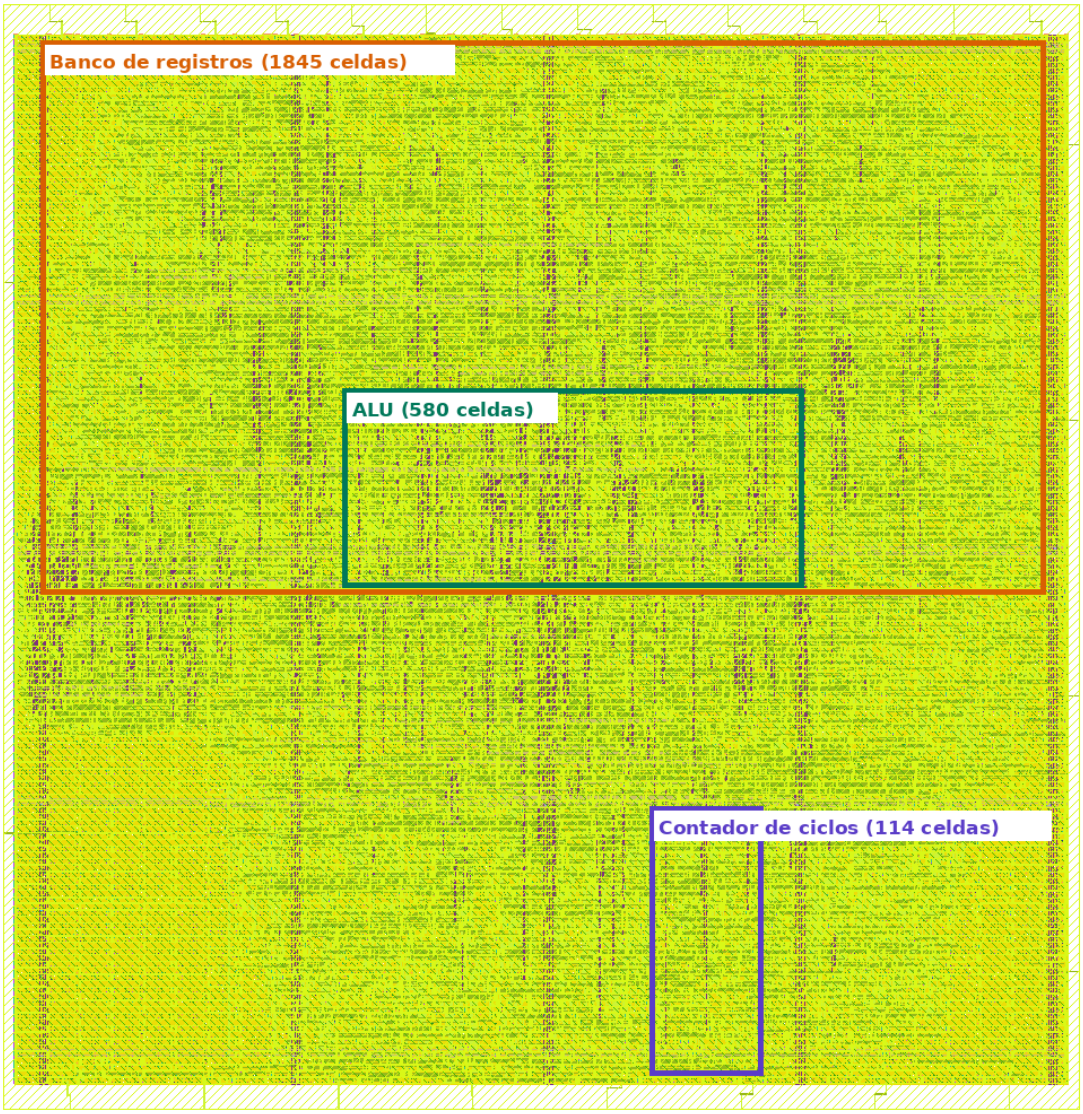


Figura 4: Layout de la corrida final en KLayout con el register file, la ALU y el contador de ciclos marcados.

Fuente: Elaboración propia.

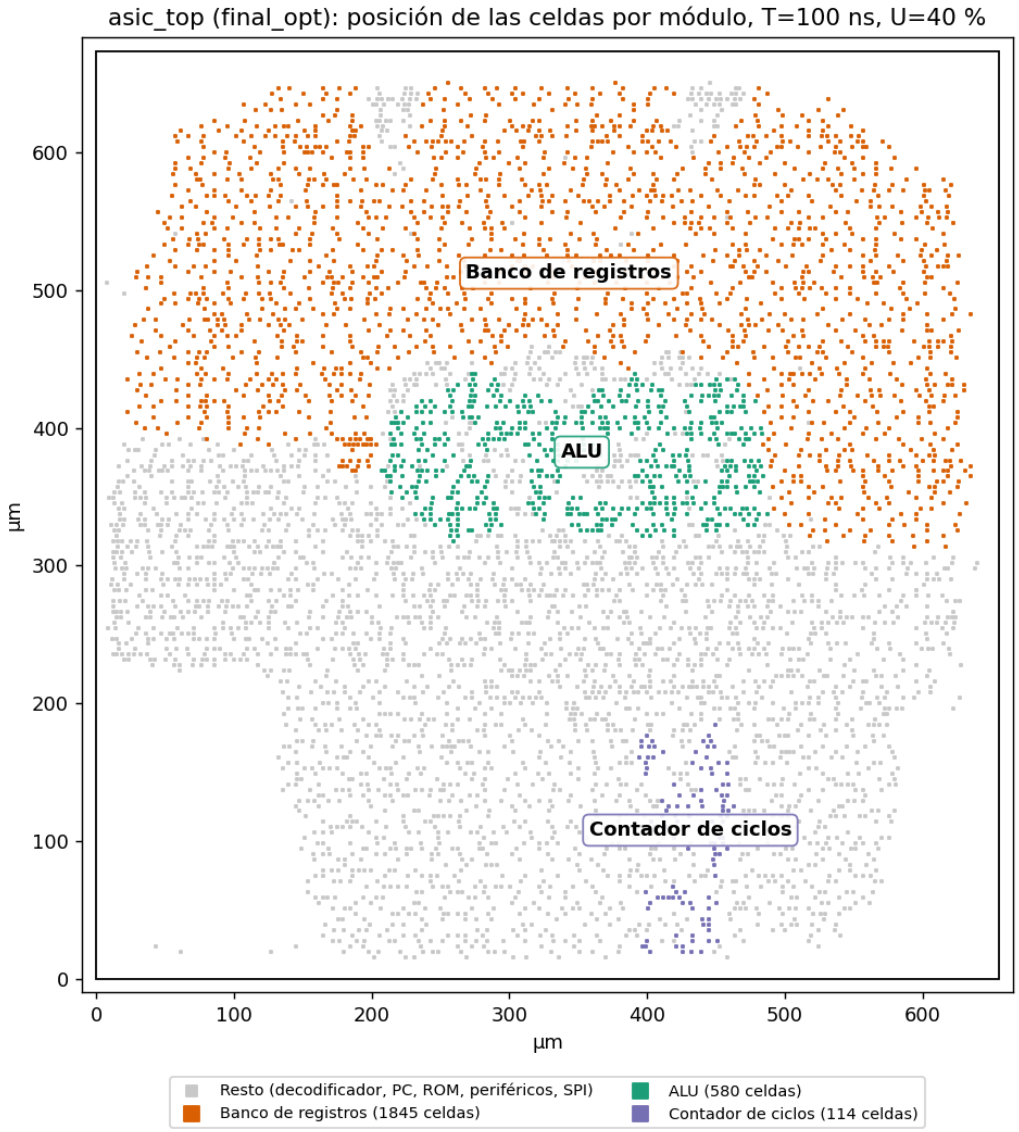


Figura 5: Ubicación de las celdas de cada módulo según su prefijo jerárquico.

Fuente: Elaboración propia.

9. Simulación funcional post-síntesis

9.1. Método

Se simuló el netlist post-síntesis (sin retardos anotados) con Verilator, usando los modelos funcionales de las celdas de GF180 y el testbench del juego, que presiona los botones en ciclos de reloj definidos y simula el rebote del pulsador. Los tres flip-flops del PDK están definidos como primitivas de usuario (UDP) que Verilator no soporta, por lo que se escribieron modelos funcionales propios

(`ff_gf180_modelos.v`); las celdas combinacionales se tomaron del PDK, excluyendo las de tipo *hold* y de fuerza débil.

Espera acortada. Con las constantes reales, la espera de tres segundos son $3 \cdot 10^7$ ciclos, impracticables de simular. Se sintetizó una variante (`sim_corto`) con constantes cortas, usada en **ambas** simulaciones (RTL y netlist), como permite el enunciado, que indica que se *puede* acortar la espera (Cuadro 15). El netlist de esta variante tiene 5.644 celdas y 164.585 μm^2.

Por qué se simula `sim_corto` y no `final_opt`. El netlist de `final_opt` lleva las constantes de 10 MHz del juego real (30.000.000 ciclos de espera), que el testbench no puede esperar. `sim_corto` es solo la síntesis (sin place and route) del mismo RTL que `final_opt`, incluido el comparador de saltos, y **lo único que difiere entre ambos es la ROM de constantes**.

Cuadro 15: Constantes cortas usadas en las simulaciones.

Constante	Ciclos
Espera de 3 s	20.000
Una décima	1.000
Antirrebote	200
Parpadeo	2.000

Fuente: Elaboración propia.

9.2. Resultados

El netlist simulado es `resultados/sim_corto/sintesis/asic_top.nl.v`, copiado al repositorio porque `librelane/runs/` está en `.gitignore`; se ejecutó con `make sim-post NETLIST=` apuntando a ese archivo. El testbench juega 10 rondas, incluyendo una con un botón incorrecto y otra con un botón sostenido. Resultado:

- `JUEGO: OK`, 0 fallos y 66 de 66 eventos verificados.
- Trazas de la simulación RTL y del netlist: **idénticas** (mensaje `TRAZAS IGUALES`).

Ronda correcta. Los estímulos se aplican en los mismos ciclos en ambas simulaciones y las trazas de LEDs y displays coinciden ciclo a ciclo. El tiempo mostrado (en décimas) y el número de ciclos medidos son por lo tanto iguales en el RTL y en el netlist. En la primera ronda (reacción de 3 décimas, TENTH = 1.000 ciclos) el LED objetivo se enciende en el ciclo 24.289 y el display cambia a `3` (`1001111` en `seg2`) en el ciclo 27.873, es decir, **3.584 ciclos medidos** después del LED; ese valor es el mismo en el RTL y en el netlist. La traza de la primera ronda en ambas simulaciones es:

```
TRACE cyc=0     led=0000 seg1=0111111 seg2=0111111
TRACE cyc=287   led=0101 seg1=0111111 seg2=0111111
TRACE cyc=2303  led=1010 seg1=0111111 seg2=0111111
TRACE cyc=3602  led=1111 seg1=0111111 seg2=0111111
TRACE cyc=24289 led=0100 seg1=0111111 seg2=0111111
TRACE cyc=27873 led=0100 seg1=0111111 seg2=1001111
TRACE cyc=27887 led=1111 seg1=0111111 seg2=1001111
```

Código 3: Inicio de la traza (RTL y netlist, idénticas).

Pulsación incorrecta. El testbench aplica un botón distinto del LED objetivo y verifica que los displays muestren `EE` y que la ronda se reinicie (comprobación `"boton incorrecto: los displays no muestran EE"`). Presionar durante la fase de los cuatro LEDs encendidos no se considera un error, y esa condición también se verifica. Esta regla se cumple tanto en el RTL como en el netlist, por lo que ambos reinician la ronda con el mismo comportamiento. La traza de la ronda 3 (la que lleva el botón incorrecto) es la siguiente: el LED objetivo aparece en el ciclo 76.913; 1.020 ciclos después (una décima de espera del testbench más la pulsación y su detección) los dos displays pasan a `EE` (`1111001` en `seg1` y `seg2`); a los 7 ciclos los cuatro LEDs vuelven a encenderse (la ronda se reinicia), y el nuevo LED objetivo aparece en el ciclo 98.643, 20.703 ciclos después, es decir, tras una nueva fase de 4 LEDs de al menos 20.000 ciclos. La ronda repetida termina con la reacción de 12 décimas (`12` en los displays, ciclo 111.317).

```
TRACE cyc=76913  led=0100 seg1=0111111 seg2=0000111
TRACE cyc=77933  led=0100 seg1=1111001 seg2=1111001
TRACE cyc=77940  led=1111 seg1=1111001 seg2=1111001
TRACE cyc=98643  led=1000 seg1=1111001 seg2=1111001
TRACE cyc=111317 led=1000 seg1=0000110 seg2=1011011
```

Código 4: Traza de la ronda con botón incorrecto (RTL y netlist, idénticas).

10. Conclusiones

- El Pochoco SoC con el juego de reflejos llegó a un GDSII con **DRC, LVS y antenas limpios** y con setup y hold cumplidos en los nueve corners ($T = 100$ ns, $U = 40\,\%$). El die ocupa 0,4416 mm^2, con un costo equivalente de US$10.156.
- Las construcciones de FPGA más peligrosas fueron las que **no producen mensajes**: la inicialización con `$readmemh` y las constantes de reloj. Su detección requirió revisar el RTL y no solo los logs.

- La estrategia de memoria fue la decisión de mayor impacto: la ROM combinacional redujo el área, el costo y la longitud de cable en un factor de $\sim$4 respecto de la memoria unificada de flip-flops, y es la única de las dos que cerró timing. La estrategia 2 no se completó (Sección 4).
- El cierre de timing tiene un margen amplio: se cumple hasta $T = 70$ ns (14,3 MHz) y el cambio de RTL ganó 2,35 ns con un $+1\,\%$ de área. La ruta crítica está dominada por celdas de retardo insertadas por el *resizer*, de modo que mejoras mayores requerirían controlar ese comportamiento.
- La simulación del netlist coincide ciclo a ciclo con el RTL, lo que valida que la síntesis conserva el comportamiento del juego.
- Limitaciones: el barrido de reloj se hizo con el RTL base y solo el punto de 100 ns con el RTL modificado; el costo se estima con una relación lineal con el área; y la simulación post-síntesis es funcional, sin retardos.

Referencias

[1] LibreLane. *LibreLane documentation*. https://librelane.readthedocs.io/en/stable/

[2] GlobalFoundries. *GF180MCU PDK documentation*. https://gf180mcu-pdk.readthedocs.io/en/latest/

[3] The OpenROAD Project. *OpenROAD documentation*. https://openroad.readthedocs.io/en/latest/

[4] J. Gómez Mir. *Proyecto 3: Arquitectura de Computadores*, Universidad de los Andes, 2026-2.

[5] A. Waterman y K. Asanović (eds.). *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. RISC-V International.

[6] N. Villegas. *Pochoco SoC* (repositorio base del curso). https://github.com/nic0villegasc/pochoco_soc

[7] Grupo 2. *Proyecto 3 Arquitectura de Computadores*. https://github.com/F-Izcue-223/Proyecto-3-Arquitectura-de-Computadores

Declaración de Uso de Herramientas de Inteligencia Artificial

1. Identificación de la actividad

- **Curso:** Arquitectura de Computadores
- **Código del curso:** 433
- **Nombre de la actividad:** Proyecto Nº3
- **Fecha de entrega:** 4 de noviembre de 2026
- **Nombre(s) de(los) estudiante(s):** Josefa Alcaíno Lira, Felipe Izcúe Ferrer, Martín Norton Haigis

2. Declaración sobre uso de IA

☐ No se utilizaron herramientas de inteligencia artificial en ninguna etapa del desarrollo de esta actividad.
☒ Sí se utilizaron herramientas de inteligencia artificial, según se detalla en la sección 3 de este documento.

3. Descripción de uso de IA

Herramienta empleada	Claude (Anthropic): Sonnet 5.5
Fechas de uso	Entre el 20 de octubre y el 3 de noviembre de 2026
Estudiantes	Josefa Alcaíno Lira, Felipe Izcúe Ferrer y Martín Norton Haigis
Propósito del uso	Adaptación del RTL de FPGA a ASIC (Verilog), extensión del ensamblador en Python y generación de la ROM del juego, y creación de testbenches y de la simulación post-síntesis. Creación de las configuraciones y scripts de LibreLane para las corridas, los barridos de reloj y de utilización, y los experimentos de timing. Análisis de los reportes de síntesis, timing, DRC, LVS y antenas, y elaboración de las figuras (Python). Explicación de los resultados y del flujo para la defensa. Apoyo en la escritura del informe en LaTeX a partir del repositorio y de los resultados de las corridas, y en la redacción del README y la organización del repositorio.
Descripción de integración	Claude se usó de forma iterativa: el grupo planteaba el problema, Claude escribía el código, el script o el texto, y el grupo lo revisaba, lo ejecutaba y pedía correcciones. Claude escribió el RTL de ASIC, la extensión del ensamblador, los testbenches, la simulación post-síntesis y las configuraciones y scripts de LibreLane. El grupo ejecutó todas las corridas en sus computadores y verificó los resultados; Claude leyó los reportes para armar las tablas y gráficos, y redactó el informe, el README y la guía de la defensa. Todo el contenido fue revisado por el grupo, que es responsable de poder explicarlo y defenderlo.

4. Declaración de responsabilidad académica

Los firmantes de este documento declaramos que:

- Hemos revisado críticamente todo el contenido entregado.
- Comprendemos plenamente el trabajo presentado y somos capaces de explicarlo y defenderlo académicamente.
- Asumimos responsabilidad por la calidad, veracidad y originalidad del contenido entregado, independientemente del uso o no uso de herramientas de inteligencia artificial.
- La información declarada en este documento es veraz.







Josefa Alcaíno Lira	Felipe Izcúe Ferrer	Martín Norton Haigis